Assignment 4: Storage, I/O, File Systems

Suhaib Abdi Muhummed, muhummed@kth.se
Abdulaziz Hassan Farah, ahfarah@kth.se
Daniel Sherif, dsherif@kth.se

Question 1: HDD Scheduling

For a disk drive of 3000 cylinders as illustrated below. Assume the disk arm is now serving a request at 1800. Before that, it just served a request at 1600.

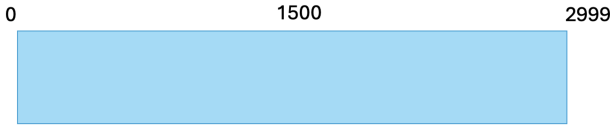

Figure 1

Given a queue of pending requests {1912, 260, 420, 280, 2023, 2788, 2901, 1400, 1600, 1501}, what is the total head movement (in cylinders) for serving all these pending requests, if using each of the following scheduling algorithms?

1. Shortest Seek Time First
2. SCAN
3. C-SCAN
4. FCFS
5. Which algorithm gives the best performance? Please briefly explain why.

Answer:

The disk has cylinders numbered from 0 to 2999. The current head position is 1800, and the previous position was 1600, therefore the head is moving in the direction of increasing cylinder numbers.

1. **Shortest Seek Time First (SSTF)**

   The request closest to the current head position is always selected.

   Service order:

   $1600 \rightarrow 1800 \rightarrow 1912 \rightarrow 2023 \rightarrow 1600 \rightarrow 1501 \rightarrow 1400 \rightarrow 420 \rightarrow 280 \rightarrow 260 \rightarrow 2788 \rightarrow 2901$

Total head movement:

$$
\begin{aligned}
&|1600-1800|+|1800-1912|+|1912-2023|+|2023-1600|+|1600-1501| \\
&+|1501-1400|+|1400-420|+|420-280|+|280-260|+|260-2788|+|2788-2901| \\
&=200+112+111+423+99+101+980+140+20+2528+113 \\
&=\boxed{4827}
\end{aligned}
$$

2. **SCAN**

   The head continues in the current direction until it reaches the end of the disk, then reverses direction.

   Service order:

   $$1600 \to 1800 \to 1912 \to 2023 \to 2788 \to 2901 \to 2999 \to 1501 \to 1400 \to 420 \to 280 \to 260$$

   Total head movement:

   $$
   \begin{aligned}
   &200+112+111+765+113+98+1498+101+980+140+20 \\
   &=\boxed{4138}
   \end{aligned}
   $$

3. **C-SCAN**

   The head moves in one direction only. After reaching the end of the disk, it jumps to the beginning and continues servicing requests.

   Service order:

   $$1600 \to 1800 \to 1912 \to 2023 \to 2788 \to 2901 \to 2999 \to 0 \to 260 \to 280 \to 420 \to 1400 \to 1501$$

   Total head movement:

   $$
   \begin{aligned}
   &200+112+111+765+113+98+2999+260+20+140+980+101+99 \\
   &=\boxed{5998}
   \end{aligned}
   $$

4. **FCFS (First-Come, First-Served)**

   Requests are served in the order they arrive.

   Service order:

   $$1600 \to 1800 \to 1912 \to 260 \to 420 \to 280 \to 2023 \to 2788 \to 2901 \to 1400 \to 1600 \to 1501$$

   Total head movement:

   $$
   \begin{aligned}
   &200+112+1652+160+140+1743+765+113+1501+200+99 \\
   &=\boxed{6785}
   \end{aligned}
   $$

5. **Best Performance**

   The **SCAN algorithm** provides the best performance in this case, as it results in the minimum total head movement. SCAN reduces unnecessary long jumps by servicing requests in one direction before reversing, leading to more efficient disk access.

Question 2: NVM Garbage Collection

NAND flash memory controller optimizes scheduling to improve wear-leveling by garbage collection. As illustrated below, flash controller tracks 4 blocks, each consisting of 8 pages. Green = valid, red = invalid, white = empty. Assume you have an infinite staging space for merging blocks. Each request either reads or write a page. Assume reading a page takes 10 time latency, writing/copying a page takes 20 time latency, and erasing a block takes 100 time latency.

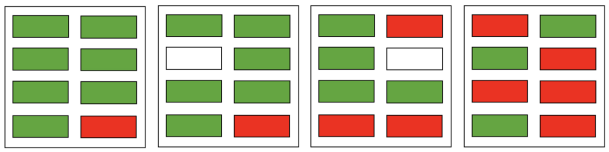

Figure 2

1. Given a queue of pending requests, S1 = {read, read, write, write, read, write, read, read}. Please list the minimum latency needed for each request in the queue. Please also briefly illustrate how to reach those latencies.

2. Give a queue of pending requests, S2 = {write, write, write, write, read, write, read, read}. Please list the minimum latency needed for each request in the queue. Please also briefly illustrate how to reach those latencies.

Answer:

The read latency is 10 time units per page, the write (or copy) latency is 20 time units per page, and erasing a block takes 100 time units. There are 4 blocks, denoted as $B0, B1, B2, B3$, where $B0$ is the leftmost block and $B3$ is the rightmost block.

1. **Queue** $S1 = \{R1, R2, W1, W2, R3, W3, R4, R5\}$

   Block $B0$ has the smallest wear, since it contains the fewest invalid pages. Therefore, the first read request $R1$ is served from $B0$ with a latency of 10, giving a total latency of 10. The second read request $R2$ can also be served from $B0$, with latency 10, resulting in a total latency of $10 + 10 = 20$.

   Block $B1$ contains one free page, so the write request $W1$ is placed in $B1$ with a latency of 20. The total latency becomes $20 + 20 = 40$. Block $B2$ also has one

free page, therefore $W2$ is placed in $B2$ with latency 20, giving a total latency of $40 + 20 = 60$.

The next request $R3$ is a read, which can be served from any block that contains valid pages. Its latency is 10, and the total latency becomes $60 + 10 = 70$.

When request $W3$ arrives, there are no free pages left in any block. Therefore, garbage collection (GC) is triggered. To minimize the amount of data copied, block $B3$ is selected since it contains the largest number of invalid pages. According to the figure, block $B3$ contains 3 valid pages.

Each valid page must be read and copied, which costs $10 + 20 = 30$ time units per page. Thus, copying the valid pages costs $3 \times 30 = 90$, and erasing the block costs an additional 100. The total GC latency is therefore $90 + 100 = 190$.

After garbage collection, the total latency becomes $70 + 190 = 260$.

Finally, the read requests $R4$ and $R5$ can be served from any block with valid pages. Each read takes 10 time units, so the final total latency for $S1$ is:

$$260 + 10 + 10 = 280.$$

Per-request latency for $S1$:

- $R1 = 10$
- $R2 = 10$
- $W1 = 20$
- $W2 = 20$
- $R3 = 10$
- $W3 = 190$
- $R4 = 10$
- $R5 = 10$

Total latency for $S1$ is 280.

2. **Queue** $S2 = \{W1, W2, W3, W4, R1, W5, R2, R3\}$

   After completing $S1$, queue $S2$ starts from the flash state left by $S1$. After garbage collection in $S1$, block $B3$ contains one valid page (from $W3$) and seven free pages.

   Write request $W1$ is placed in $B3$ with latency 20, giving a total latency of $280+20 = 300$. Requests $W2$, $W3$, and $W4$ are also written to block $B3$, each with latency 20, increasing the total latency to 360.

   The read request $R1$ can be served from any block with valid pages and takes 10 time units, resulting in a total latency of 370.

The write request $W5$ is written to block $B3$, which still has free pages available. This write takes 20 time units, increasing the total latency to 390.

Finally, the read requests $R2$ and $R3$ are served, each with latency 10. The final total latency becomes:

$$390 + 10 + 10 = 410.$$

Per-request latency for $S2$:

- $W1 = 20$
- $W2 = 20$
- $W3 = 20$
- $W4 = 20$
- $R1 = 10$
- $W5 = 20$
- $R2 = 10$
- $R3 = 10$

The total latency after executing both $S1$ and $S2$ is 410 time units.

Question 3: RAID Levels

The reliability of a storage device is often measured by MTBF (the mean time to failure). Given an array of N disks, assume their failures are independent. The mean time to failure for N disks is often modelled as (MTBF / N). Assume a single disk may have a failure every 1000000 hours.

Disk 0	Disk 1	.....	Disk (N-1)
block 0	block 1		P0

Figure 3

1. Assume RAID Level 0 is used. What is the highest level of parallelism (defined as the number of blocks accessed in parallel) you can achieve while still maintaining the mean time to failure longer than 120000 hours? Explain your answer. Hint: first determine N.

2. With N disks in use (N from (a)). Assume RAID Level 4 is used. Assume a large write request of 1 MB data arrives, and a block size of 64KB is used. Fill up in the table above (add more rows / columns if needed). Assume the write bandwidth to one disk is 100MB/s. What is the peak achievable write bandwidth for this request? Explain your answer.

Answer :

1. The mean time to failure (MTBF) of a single disk is given as 1,000,000 hours. Fr RAI level 0, there is no redundancy, and the failure of any single disk causes the entire array to fail. Assuming independent disk failures, the MTBF of a RAID 0 array with N disks is:

$$\text{MTBF}_{RAID0} = \frac{\text{MTBF}_{disk}}{N}$$

We want the MTBF of the RAID 0 array to be longer than 120,000 hours. Therefore,

$$\frac{1{,}000{,}000}{N} \geq 120{,}000$$

Solving for N gives:

$$N \leq \frac{1{,}000{,}000}{120{,}000} \approx 8.33$$

Since the number of disks must be an integer, the maximum possible number of disks is $N = 8$.

In RAID level 0, the level of parallelism is equal to the number of disks, since blocks can be accessed in parallel across all disks. Therefore, the highest level of parallelism that still maintains a mean time to failure longer than 120,000 hours is: $\boxed{8}$

2. Given:

   - Number of disks $N = 8$ (from part 1)
   - RAID 4: 7 data disks (Disk 0âĂŞ6) and 1 parity disk (Disk 7)
   - Write request: 1 MB of data
   - Block size: 64 KB
   - Write bandwidth per disk: 100 MB/s

   Step 1: Determine number of blocks and stripes

   - Data size: 1 MB = 1024 KB
   - Number of data blocks: $\frac{1024}{64} = 16$ blocks
   - Each stripe in RAID 4 contains 7 data blocks (one per data disk) and 1 parity block.
   - Full stripes: $\lfloor \frac{16}{7} \rfloor = 2$ full stripes (14 data blocks)
   - Partial stripe: remaining 2 data blocks (blocks 14 and 15)
   - Total stripes: 3 (stripe 0, stripe 1, stripe 2)

Step 2: Table of writes

Table 1Table of writes across disks (data blocks and parity).

Disk 0	**Disk 1**	**Disk 2**	**Disk 3**	**Disk 4**	**Disk 5**	**Disk 6**	**Disk 7**
block0: data0	block0: data1	block0: data2	block0: data3	block0: data4	block0: data5	block0: data6	block0: P0
block1: data7	block1: data8	block1: data9	block1: data10	block1: data11	block1: data12	block1: data13	block1: P1
block2: data14	block2: data15						block2: P2

Note: Empty cells indicate no write for that block index.

Step 3: Write operations and timing

- Full stripes (stripe 0 and stripe 1): Each full stripe write writes 7 data blocks and 1 parity block in parallel to all 8 disks. Time per full stripe = time to write one block (64 KB) at 100 MB/s:

$$t_{\text{block}} = \frac{64 \text{ KB}}{100 \text{ MB/s}} = \frac{0.0625 \text{ MB}}{100 \text{ MB/s}} = 0.000625 \text{ s}$$

  Time for two full stripes: $2 \times 0.000625 = 0.00125$ s.

- Partial stripe (stripe 2): Involves a read-modify-write cycle:
  - Read old data from 5 non-updated data disks and old parity from Disk 7 (6 reads total).
  - Compute new parity.
  - Write 2 new data blocks and new parity (3 writes total).
  - Reads and writes are done in parallel across disks, so:

$$t_{\text{partial}} = t_{\text{read}} + t_{\text{write}} = 0.000625 + 0.000625 = 0.00125 \text{ s}$$

  - Total time:

$$t_{\text{total}} = 0.00125 + 0.00125 = 0.0025 \text{ s}$$

Step 4: Peak write bandwidth

$$\text{Bandwidth} = \frac{\text{Data size}}{\text{Total time}} = \frac{1 \text{ MB}}{0.0025 \text{ s}} = 400 \text{ MB/s}$$

Question 4: Allocation Methods

Assume a file system uses 512 bytes for logical blocks and 512 bytes for physical blocks. Assume free device blocks are tacked by a bit vector that is used for implementing the free-space list, and '1' means the device block is free. In this exercise, First Fit is used when requesting a new block from the free-space list {00111100111111111000000101010}.

Given the following list of files {File0, 1024 bytes}, {File1, 500 bytes}, {File2, 128 bytes}, {File3, 4096 bytes},

(a) Use the contiguous allocation strategy, specify the logical-to-physical address mapping for each file. Your output should look like Figure 5.

File Name	Start (physical block)	List of physical blocks
File0		
File1		
File2		
File3		

Figure 4

(b) Use the linked allocation, assume the block pointer needs 8 bytes. Specify the logical-to-physical address mapping for each file. Your output should look Figure 6.

File Name	Start (physical block)	Linked List of physical blocks
File0		
File1		
File2		
File3		

Figure 5

(c) Use the indexed allocation, specify the logical-to-physical address mapping for each file. The output should look like Figure 6.

File Name	Index Block Id	List of blocks in Index Block
File0		
File1		
File2		
File3		

Figure 6

(d) Calculate the fragmentation ratio for each file using the equation = (File Size)/(Allocated Physical Blocks*Block Size) for each allocation method. The output should look Figure 7.

File Name	Contiguous	Linked	Indexed
File0			
File1			
File2			
File3			

Figure 7

(e) If we are currently at logical block 2 of File 3 and want to access its logical block 4. For each allocation method, how many physical blocks must be read from the disk? Please briefly explain why.

Answer: a. See Table 1.

Table 2: Contiguous Allocation Mapping

File Name	Start (Physical Block)	List of Physical Blocks
File0	2	2, 3
File1	4	4
File2	5	5
File3	8	8, 9, 10, 11, 12, 13, 14, 15

Table 3: Linked Allocation Mapping

File Name	Start (Physical Block)	Linked List of Physical Blocks
File0	2	2, 3, 4
File1	5	5
File2	8	8
File3	9	9, 10, 11, 12, 13, 14, 15, 16, 22

b. Assuming pointer overhead reduces usable block capacity, meaning we now use 508 bytes per block, we answer as in Table 2. See Table 2.

c. See Table 3.

Table 4: Indexed Allocation Mapping

File Name	Index Block ID	List of Blocks in Index Block
File0	2	3, 4
File1	5	8
File2	9	10
File3	11	12, 13, 14, 15, 16, 22, 24, 26

d. See Table 4.

Table 5: Fragmentation Ratio

File Name	Contiguous	Linked	Indexed
File0	1.00	0.667	0.667
File1	0.98	0.98	0.488
File2	0.25	0.25	0.125
File3	1.00	0.889	0.889

e. In contiguous allocation methods, we can calculate the physical address of the block we want with a single read. This is because all blocks are contiguous, meaning the system finds the block 2 steps ahead of block 2 by determining the offset and adding that to the physical address.

In linked allocation methods, you've got to read each file in sequence. This is because linked allocation method is pointer based in the sense that every "step" requires the

pointer of the next block. So each block contains a pointer to the next block. Block 3 points to 4 and block 4 to 5.

In indexed allocation methods, there are 2 reads. This is because there is a pointer index table block that contains pointers to all data blocks of the file, so you first read this table block and then jump to the desired block.

All in all, for contiguous, 1 block read. For linked, 3 blocks read and for indexed, 2 blocks read.

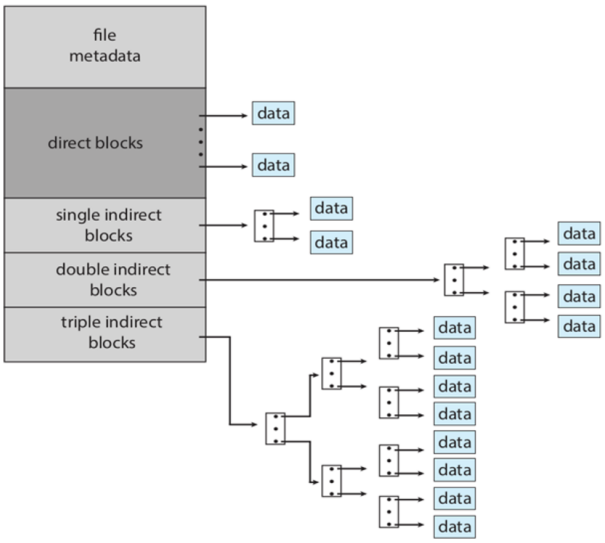

Figure 8

Question 5: File System Structures

In a file system, where each file has a per-file FCB to represent the file and a FCB is implemented as an inode entry like below. Indexed Allocation with Combined scheme is used. 15 pointers to blocks are kept in the file's inode. Assume that each data block on the disk is 8 KB in size, and a pointer to a disk block is 8 bytes. Assume the inode is already copied in memory.

- The first 12 pointers directly point to data blocks.
- The 13th pointer points to a single index block, which contains a list of pointers to data blocks.
- The 14th pointer points to a double indirect block, which contains a list of pointers to single index block.
- The 15th pointer points to a triple indirect block, which contains a list of pointers to double index block.

Answer the following questions:

1. Explain and calculate the size of data that can be reached with exactly one disk I/O operation?
2. Explain and calculate the size of data that can be reached with exactly two disk I/O operations?
3. Explain and calculate the size of data that can be reached with exactly three disk I/O operations?

4. Explain and calculate the size of data that can be reached with exactly four disk I/O operations?

Answer:

Block size = 8 KB

Pointer size = 8 bytes

Pointers per block: 8192 bytes / 8 bytes = 1024 pointers/block

Note: Inode is already in memory, so accessing the inode themselves expends no disk I/Os

1. Direct pointers are directly accessible since the inode is already in memory. Therefore, it only requires 1 disk access, the data block itself. There are 12 direct pointers, and each pointer points to one data block.

12 * 8 KB = 96 KB.

There is 96 KB of data reachable.

2. 1st I/O loads an index block into memory. 2nd I/O reads a data block. This results in 1024 pointers to data blocks, meaning 1024 * 8 KB = 8192 KB = 8 MB

3. 1st I/O reads double indirect block, 2nd reads the following single indirect and 3rd I/O reads the data block itself.

So 1024 * 1024 * 8 KB = 8 GB. 8 GB of data accessible with 3 I/O reads.

4. 1st would read the triple indirect, which again would result in 1024 more pointers. Successively like previously the double, single indirect and finally the pointer to data itself is also accessible.

This results in 1024*1024*1024*8 KB = 8 TB of accessible data.

Programming Section

Question 6: File Directories

Acyclic-graph directories support files to be shared. Soft and Hard links can be used. In this experiment, first, create a new file called file_original.txt with random text. Answer the following questions.

- (a) Specify the command did you use to create a soft link that creates a new file called 'file_soft.txt' pointing to 'file_original.txt'.
  - ln -s file_original.txt file_soft.txt
- (b) Specify the command that can be used to find the inode number of a file. Are the inodes the same for 'file_soft.txt' and 'file_origin.txt' in your experiment, and why? You can report screenshots.
  - Command to find inode number: ls -li file_original.txt file_soft.txt
    - 17304024 -rw-rw-r– 1 suhkth suhkth 78 Dec 16 14:24 file_original.txt
    - 17305507 lrwxrwxrwx 1 suhkth suhkth 17 Dec 16 14:24 file_soft.txt -> file_original.txt
  - The inodes are different because a soft link is a special file that contains a path reference to the target file. The soft link has its own inode that stores the pathname of the original file, not the actual file data. The soft link is essentially a separate file that points to another file's path."a special file that contains a path reference to the target file. The soft link has its own inode that stores the pathname of the original file, not the actual file data. The soft link is essentially a separate file that points to another file's path.
- (c) Now edit the contents of 'file_soft.txt'. Have the contents of 'file_origin.txt' been altered as well? Explain your observation.
  - Yes, the contents of file_original.txt have been altered as well. When you edit file_soft.txt, the operating system follows the symbolic link to the actual file (file_original.txt) and modifies that file's content. The soft link doesn't contain the data itself. It's just a pointer to the original file's path. Therefore, any read or write operations on the soft link are transparently redirected to the target file.
- (d) Delete 'file_origin.txt' with command 'rm'. Can you still edit "file_soft.txt'? Explain your observation.
  - You can not still edit file_soft.txt. After deleting file_original.txt, the soft link file_soft.txt becomes a dangling link (also called a broken link). The soft link

still exists and still contains the pathname "file_original.txt", but that pathname no longer points to any existing file. When you try to access file_soft.txt, the system attempts to follow the link to file_original.txt, but since that file no longer exists, the operation fails with "No such file or directory". The soft link itself exists (you can see it with ls -l), but it's useless because its target has been deleted. This demonstrates a key limitation of soft links: they are path-based and can become broken if the target is moved or deleted.

Now, create 'file_origin2.txt' with random text. Answer the following questions.

- (e) Specify the command did you use to create a hard link that creates a new file called 'file_hard.txt' pointing to 'file_origin2.txt'.
  - ln file_origin2.txt file_hard.txt
- (f) Are the inodes the same for 'file_hard.txt' and 'file_origin2.txt', and why? You can report screenshots.
    - 17306202 -rw-rw-r– 2 suhkth suhkth 83 Dec 16 14:24 file_hard.txt
    - 17306202 -rw-rw-r– 2 suhkth suhkth 83 Dec 16 14:24 file_origin2.txt
  - The inodes are the same because a hard link creates another directory entry (filename) that points to the exact same inode as the original file. Both files are essentially equal references to the same underlying data on disk. There is no "original" and "copy" - both names are equal pointers to the same file data.
- (g) Edit the contents of 'file_hard.txt'. Have the contents of 'file_origin2.txt' been altered as well? Are the inodes for 'file_hard.txt' and 'file_origin2.txt' still the same? Explain your observation.
  - Yes, the contents of file_origin2.txt have been altered as well.
  - Yes, the inodes for both files are still the same (17306202)
  - Since both file_hard.txt and file_origin2.txt point to the same inode (same physical location on disk), they are essentially two names for the exact same file. When you modify one, you're modifying the actual data blocks associated with that inode. Therefore, any changes made through one filename are immediately visible through the other filename because they both access the same underlying data. The inodes remain the same after editing because hard links always point to the same inode - that's their fundamental characteristic. The file content is stored in the inode's data blocks, and editing doesn't change which inode the filenames point to.
- (h) Delete 'file_origin2.txt' with command 'rm'. Can you still edit 'file_hard.txt'? Explain why.

– Yes, you can still read and edit file_hard.txt after deleting file_origin2.txt. This is because hard links work fundamentally differently from soft links. When you use rm to delete a file, you're not actually deleting the file data. You're removing a directory entry (decrementing the link count). The actual file data in the inode is only deleted when the link count reaches zero. In this case:

  1. Initially, both file_origin2.txt and file_hard.txt pointed to inode 17306202 (link count = 2)
  2. After rm file_origin2.txt, only the directory entry for file_origin2.txt was removed
  3. The inode 17306202 still exists with link count = 1 (because file_hard.txt still points to it)
  4. The actual file data remains accessible through file_hard.txt

  The file system will only delete the actual data blocks when the last hard link is removed (link count becomes 0). This makes hard links much more robust than soft links. They maintain access to the file data as long as at least one hard link exists.

Answer:

Question 7: I/O System Calls

You will create a simple C program that uses multiple threads to read or write from a file.ÂăYou are free to modify the **Download code skeleton**. The program initially allocates a memory buffer of N bytes, where N is set by the user from argv[1]. Initialize the buffer with some values. The program then creates a file to store the values.

Your program should create P threads using pthread_create(), where P is set by the user from argv[2]. Each thread will write a portion from the buffer to the file. You will decide how to split the list of requests among threads. Add timers so that you can calculate write bandwidth = Bytes_Written / Elapsed_Time.

Next, the program should re-open the file. Now use P threads to read a portion from the file. Add timers so that you can calculate read bandwidth. An example output is shown below

```
$./io_perf 1000000 2
List1: Write xxx bytes, use 2 threads, elapsed time xxx s, write bandwidth: xxx MB/s
List1: Read xxx bytes, use 2 threads, elapsed time xxx s, read bandwidth: xxx MB/s

List2: Write xxx bytes, use 2 threads, elapsed time xxx s, write bandwidth: xxx MB/s
List2: Read xxx bytes, use 2 threads, elapsed time xxx s, read bandwidth: xxx MB/s
```

Figure 9

1. Run your program with N: 16000000, P: 1. Report your output in screenshot.

(a) ./hw4_io_perf 16000000 1

i. List1: Write 15990784 bytes, use 1 threads, elapsed time 0.008412 s, write bandwidth: 1812.89 MB/s

ii. List1: Read 15990784 bytes, use 1 threads, elapsed time 0.002206 s, read bandwidth: 6912.96 MB/s

iii. List2: Write 16000000 bytes, use 1 threads, elapsed time 0.569697 s, write bandwidth: 26.78 MB/s

iv. List2: Read 16000000 bytes, use 1 threads, elapsed time 0.072147 s, read bandwidth: 211.50 MB/s

2. Run your program with N: 16000000, P: 2. Report your output in screenshot.

(a) ./hw4_io_perf 16000000 2

i. List1: Write 15990784 bytes, use 2 threads, elapsed time 0.009009 s, write bandwidth: 1692.75 MB/s

ii. List1: Read 15990784 bytes, use 2 threads, elapsed time 0.001489 s, read bandwidth: 10241.77 MB/s

iii. List2: Write 16000000 bytes, use 2 threads, elapsed time 0.463869 s, write bandwidth: 32.89 MB/s

iv. List2: Read 16000000 bytes, use 2 threads, elapsed time 0.044349 s, read bandwidth: 344.06 MB/s

3. Run your program with N: 16000000, P: 4. Report your output in screenshot.

(a) ./hw4_io_perf 16000000 4

i. List1: Write 15990784 bytes, use 4 threads, elapsed time 0.007808 s, write bandwidth: 1953.12 MB/s

ii. List1: Read 15990784 bytes, use 4 threads, elapsed time 0.001294 s, read bandwidth: 11785.16 MB/s

iii. List2: Write 16000000 bytes, use 4 threads, elapsed time 0.591634 s, write bandwidth: 25.79 MB/s

iv. List2: Read 16000000 bytes, use 4 threads, elapsed time 0.030437 s, read bandwidth: 501.32 MB/s

4. Run your program with N: 160000000, P: 2. Report your output in screenshot.

(a) ./hw4_io_perf 160000000 2

i. List1: Write 159989760 bytes, use 2 threads, elapsed time 0.069160 s, write bandwidth: 2206.16 MB/s

ii. List1: Read 159989760 bytes, use 2 threads, elapsed time 0.013270 s, read bandwidth: 11497.97 MB/s

iii. List2: Write 160000000 bytes, use 2 threads, elapsed time 3.474827 s, write bandwidth: 43.91 MB/s

iv. List2: Read 160000000 bytes, use 2 threads, elapsed time 0.307651 s, read bandwidth: 495.98 MB/s

5. What are your main observations from a-d, and explanation for the observations?

Performance Analysis and observations

Table 6Summary Table

Test	N	P	Seq W	Seq R	Rand W	Rand R
1	16M	1	1812.89	6912.96	26.78	211.50
2	16M	2	1692.75	10241.77	32.89	344.06
3	16M	4	1953.12	11785.16	25.79	501.32
4	160M	2	2206.16	11497.97	43.91	495.98

1. Sequential vs. Random Access

Sequential access is **dramatically faster** than random access in all tests, with performance gains ranging from **23x to 76x**.

Sequential/Random ratios:

- Test 1 (1 thread): Write 67.7x, Read 32.7x
- Test 2 (2 threads): Write 51.5x, Read 29.8x
- Test 3 (4 threads): Write 75.7x, Read 23.5x
- Test 4 (2 threads, 160M): Write 50.2x, Read 23.2x

This is primarily due to disk seek overhead in random access, while sequential access benefits from OS prefetching, read-ahead, cache locality, and request coalescing, allowing the I/O scheduler to issue larger, more efficient operations.

2. Read vs. Write Performance

Read operations consistently outperform writes across all workloads.

- **Sequential reads** are **3.8x-6.0x faster** than writes
- **Random reads** are **7.9x-19.4x faster** than writes

Reads benefit heavily from the OS page cache and do not require durability guarantees. Writes incur additional overhead due to metadata updates, journaling (e.g., ext4), write-through requirements, and eventual disk flushes, even when buffered.

3. Impact of Thread Count

Increasing thread count improves read performance significantly but has mixed or negative effects on writes.

- **Sequential reads** scale well: +48% (2 threads), +71% (4 threads)
- **Random reads** scale even better: +63% (2 threads), +137% (4 threads)
- **Sequential writes** show minor variation (âĹŠ7% to +8%)
- **Random writes** peak at 2 threads (+23%) and degrade at 4 threads (âĹŠ4%)

This behavior is explained by read parallelism, increased I/O queue depth, and scheduler reordering benefits. Writes suffer from serialization requirements, lock contention, metadata consistency, and context-switching overhead, which can outweigh parallelism gains.

4. Effect of Dataset Size

Larger datasets (160M vs. 16M) consistently improve performance by **12-44%** across all access patterns.

- Sequential write: +30%
- Sequential read: +12%
- Random write: +34%
- Random read: +44%

The improvement comes from amortized fixed overhead, better sustained throughput, reduced startup/termination effects, more effective scheduling, and improved cache warming over longer executions.

5. Random Access Scaling

Random reads scale strongly with threading, while random writes show diminishing or negative returns.

- **Random read scaling (16M)**:
  211 –> 344 –> 501 MB/s (1 –> 2 –> 4 threads)
- **Random write scaling (16M)**:
  26.8 –> 32.9 –> 25.8 MB/s

Schedulers can reorder random reads to minimize seek time, but random writes introduce ordering constraints, write amplification, and journaling overhead, limiting scalability and making excessive threading counterproductive.

Question 8: Memory-mapped Files

You will create a simple C program that creates file-backed memory mapping using*mmap().* First, create a file named 'file_to_map.txt' of 1MB size. Hint: this step can be performed outside your code using Linux commands. In your code, use fork() to create a child process. Both child and parent processes use *mmap* to create a memory mapping to 'file_to_map.txt'. For mmap(), child and parent processes will use shared mapping.

(a) Run your code and report the screenshot showing the virtual address returned from *mmap*. The output should look like the following

```
Child process (pid=xxx); mmap address: xxxxxxx

Parent process (pid=xxx); mmap address: xxxxxxx
```

Figure 10

Next, both child and parent processes will try to write to the file-backed memory region. The child process will write '01234' to location starting from mmaped_ptr[0]' and then read 5 characters from mmaped_ptr[4096]. The parent process will write '56789' to location starting from mmaped_ptr[4096] and then read 5 characters from mmaped_ptr[0].

(b) Report the screenshot showing the characters read by child and parent processes. The output should look like the following

```
Child process (pid=xxx); read from mmaped_ptr [4096]: xxxxx

Parent process (pid=xxx); read from mmaped_ptr [0]: xxxxx
```

Figure 11

- (c) Run your program 5-10 times. Do you always observe the same output from step (b)?
- (d) Explain how you would modify your code so that a process only read from the file after the other process has finished writing? Hint: recall thatÂă*msync()* forces data to be written to the disk and ensures the updates become visible to other processes read from the file.
- (e) Run your modified program 5-10 times and report the screenshot showing the output.

Answer:

1. ./mmap_program

(a) Child process (pid=28109); mmap address: 0x7f8d2c6b4000

(b) Parent process (pid=28108); mmap address: 0x7f8d2c6b4000

2. ./mmap_program

(a) Child process (pid=28109); read from mmaped_ptr[4096]: 56789

(b) Parent process (pid=28108); read from mmaped_ptr[0]: 01234

3. bash -c 'for i in 1..10; do echo "=== Run $i ==="; ./mmap_program; echo ""; done'

=== Run 1 ===

(a) Parent process (pid=106902); mmap address: 0x75e5f16b5000

(b) Child process (pid=106903); mmap address: 0x75e5f16b5000

(c) Parent process (pid=106902); read from mmaped_ptr[0]: 01234

(d) Child process (pid=106903); read from mmaped_ptr[4096]: 56789

=== Run 2 ===

(a) Parent process (pid=106904); mmap address: 0x7fc9c2504000

(b) Child process (pid=106905); mmap address: 0x7fc9c2504000

(c) Parent process (pid=106904); read from mmaped_ptr[0]: 01234

(d) Child process (pid=106905); read from mmaped_ptr[4096]: 56789

=== Run 3 ===

(a) Parent process (pid=106906); mmap address: 0x72850e89e000

(b) Child process (pid=106907); mmap address: 0x72850e89e000

(c) Parent process (pid=106906); read from mmaped_ptr[0]: 01234

(d) Child process (pid=106907); read from mmaped_ptr[4096]: 56789

=== Run 4 ===

(a) Parent process (pid=106908); mmap address: 0x7287ffa13000

(b) Child process (pid=106909); mmap address: 0x7287ffa13000

(c) Parent process (pid=106908); read from mmaped_ptr[0]: 01234

(d) Child process (pid=106909); read from mmaped_ptr[4096]: 56789

=== Run 5 ===

(a) Parent process (pid=106910); mmap address: 0x7620d2500000

(b) Child process (pid=106911); mmap address: 0x7620d2500000

(c) Parent process (pid=106910); read from mmaped_ptr[0]: 01234

(d) Child process (pid=106911); read from mmaped_ptr[4096]: 56789

=== Run 6 ===

(a) Parent process (pid=106912); mmap address: 0x7c86b4100000

(b) Child process (pid=106913); mmap address: 0x7c86b4100000

(c) Parent process (pid=106912); read from mmaped_ptr[0]: 01234

(d) Child process (pid=106913); read from mmaped_ptr[4096]: 56789

=== Run 7 ===

(a) Parent process (pid=106914); mmap address: 0x7868e8069000

(b) Child process (pid=106915); mmap address: 0x7868e8069000

(c) Parent process (pid=106914); read from mmaped_ptr[0]: 01234

(d) Child process (pid=106915); read from mmaped_ptr[4096]: 56789

=== Run 8 ===

(a) Parent process (pid=106916); mmap address: 0x7021d288b000

(b) Child process (pid=106917); mmap address: 0x7021d288b000

(c) Parent process (pid=106916); read from mmaped_ptr[0]: 01234

(d) Child process (pid=106917); read from mmaped_ptr[4096]: 56789

=== Run 9 ===

(a) Child process (pid=106919); mmap address: 0x734775f00000

(b) Parent process (pid=106918); mmap address: 0x734775f00000

(c) Child process (pid=106919); read from mmaped_ptr[4096]: 56789

(d) Parent process (pid=106918); read from mmaped_ptr[0]: 01234

=== Run 10 ===

(a) Parent process (pid=106920); mmap address: 0x790f56af6000

(b) Child process (pid=106921); mmap address: 0x790f56af6000

(c) Parent process (pid=106920); read from mmaped_ptr[0]: 01234

(d) Child process (pid=106921); read from mmaped_ptr[4096]: 56789

YES, the output is consistent across all 10 runs. Here's a summary:

4. The original program suffers from a race condition. Although MAP_SHARED ensures that memory updates are visible between processes, it provides no guarantee about execution order. As a result, one process may read shared memory before the other has written to it. The existing delay (e.g., 100 microsecond) only reduces the likelihood of failure-it does not ensure correctness.

   For MAP_SHARED mappings:

   - All processes share the **same physical pages**
   - Writes are **immediately visible** to other processes
   - **No system call is required** to propagate changes
   - `msync()` does **not** provide synchronization between processes using the same mapping

   What msync() actually does:

   - Flushes dirty pages to the **backing file on disk**
   - Makes updates visible to processes that later open/read the file
   - Supports persistence and file-level visibility
   - **Does not enforce ordering or coordination** between concurrently running processes

   Incorrect Approach: msync() + sleep()

   This version appears to work because `sleep()` introduces **time-based ordering**, not because of `msync()`.

   Why it's wrong:

   - Correctness relies on **timing assumptions**
   - msync() is unnecessary for shared mappings
   - sleep() wastes time and is unreliable across systems
   - The race condition is masked, not solved

   Correct Solution: Explicit Synchronization

   Proper synchronization must be enforced using **inter-process synchronization primitives**, such as **semaphores**.

   - Semaphores guarantee execution order
   - No reliance on timing
   - Correct, efficient, and portable
   - Fully resolves the race condition

5. Comparison of Synchronization Approaches

The program using msync() combined with sleep() appears to work correctly in all runs, as shown in Table below: both processes read the expected values written by the other process. However, this behavior is **not guaranteed** and occurs **by coincidence rather than by correct synchronization**.

Table 7Output of mmap_program_synced. MS= (msync + sleep), P = Parent & C = Child

Run	**mmap addr (P=C)**	**Child**	**Parent**
1	0x727da288a000	Wrote‘01234‘, MS, Read‘56789‘	Wrote‘56789‘, MS, Read‘01234‘
2	0x775a67c9f000	Wrote‘01234‘, MS, Read‘56789‘	Wrote‘56789‘, MS, Read‘01234‘
3	0x750464700000	Wrote‘01234‘, MS, Read‘56789‘	Wrote‘56789‘, MS, Read‘01234‘
4	0x701a0bd00000	Wrote‘01234‘, MS, Read‘56789‘	Wrote‘56789‘, MS, Read‘01234‘
5	0x79be91a34000	Wrote‘01234‘, MS, Read‘56789‘	Wrote‘56789‘, MS, Read‘01234‘
6	0x71346642f000	Wrote‘01234‘, MS, Read‘56789‘	Wrote‘56789‘, MS, Read‘01234‘
7	0x77a987e9c000	Wrote‘01234‘, MS, Read‘56789‘	Wrote‘56789‘, MS, Read‘01234‘
8	0x75c55e100000	Wrote‘01234‘, MS, Read‘56789‘	Wrote‘56789‘, MS, Read‘01234‘
9	0x76c363900000	Wrote‘01234‘, MS, Read‘56789‘	Wrote‘56789‘, MS, Read‘01234‘
10	0x79c29fcf1000	Wrote‘01234‘, MS, Read‘56789‘	Wrote‘56789‘, MS, Read‘01234‘

The apparent correctness is caused by the sleep() calls, which introduce artificial delays that temporarily enforce an execution order (child writes first, parent writes second). The msync() calls do not contribute to synchronization between the parent and child processes; they only flush modified memory pages to disk. Since the memory is mapped with MAP_SHARED, changes are already immediately visible between processes, making msync() unnecessary for inter-process visibility.

However, the correctness of Semaphore-Based Synchronization does not depend on timing, delays, or scheduling luck. Instead, it is guaranteed by the use of semaphores, which provide explicit process synchronization. Semaphores ensure that one process blocks until the other has completed its critical operation, eliminating race conditions. This is shown in Table below.

Table 8Repeated runs of `mmap_program_proper_sync`. Swc = Signaled write completion, Cwc = Child write confirmed.

Run	**mmap addr (P=C)**	**Child**	**Parent**
1	0x78fce3500000	Wrote'01234', Swc, Read'56789'	Cwc, Wrote'56789', Swc, Read'01234'
2	0x78413d2df000	Wrote'01234', Swc, Read'56789'	Cwc, Wrote'56789', Swc, Read'01234'
3	0x7d0501d00000	Wrote'01234', Swc, Read'56789'	Cwc, Wrote'56789', Swc, Read'01234'
4	0x703164ae4000	Wrote'01234', Swc, Read'56789'	Cwc, Wrote'56789', Swc, Read'01234'
5	0x72f39e700000	Wrote'01234', Swc, Read'56789'	Cwc, Wrote'56789', Swc, Read'01234'
6	0x72b22f839000	Wrote'01234', Swc, Read'56789'	Cwc, Wrote'56789', Swc, Read'01234'
7	0x7b282122d000	Wrote'01234', Swc, Read'56789'	Cwc, Wrote'56789', Swc, Read'01234'
8	0x7f73bf300000	Wrote'01234', Swc, Read'56789'	Cwc, Wrote'56789', Swc, Read'01234'
9	0x7c82e2300000	Wrote'01234', Swc, Read'56789'	Cwc, Wrote'56789', Swc, Read'01234'
10	0x7134c4f00000	Wrote'01234', Swc, Read'56789'	Cwc, Wrote'56789', Swc, Read'01234'